

Compress-at-the-Gate: Query-Aware Classical Prompt Compression as a Zero-Footprint Context-Engineering Primitive for LLM Gateways

Raushan

Mesh API / AI Fiesta · raushan@meshapi.ai

Author list is a placeholder for the camera-ready version.

Abstract. LLM *gateways*—unified APIs that route one request format to hundreds of backing models—sit on the critical path of a large fraction of production LLM traffic. Because a gateway touches *every* request, any context-engineering transformation it applies must be cheap in two currencies the prompt-compression literature largely ignores: *tail latency* and *accelerator footprint*. State-of-the-art neural compressors (Selective Context, LLMLingua, RECOMP) estimate token importance with a language-model forward pass, which needs its own GPU and can make end-to-end inference *slower*. We ask how far *classical*, model-free extractive compression goes at the gate. Query-aware lexical methods (BM25, centroid+MMR) retain **96.5%** of gold answers at $\sim 3\times$ compression — **$2.5\times$ the 38.7%** of a faithful neural Selective-Context baseline—while running in **0.33 ms** vs **406 ms** per request (**1213 $\times$ faster**) and holding *zero* resident model weights against the baseline's 312 MB. At the gateway, the classical/neural trade-off inverts.

1. Introduction

Building LLM applications has shifted from *prompt engineering* to *context engineering*: deciding what an agent "knows, sees, and remembers at the moment of action" [1]. A central operation is *compression*—removing low-value tokens so more of a finite, expensive context window carries signal, mitigating cost and the "lost-in-the-middle" degradation [2].

Almost all recent work frames compression as a *modeling* problem. Selective Context [3] prunes lexical units with low self-information under a small LM; LLMLingua / LongLLMLingua [4,5] use (contrastive) perplexity from a 0.5–7B LM; LLMLingua-2 [6] distills a token classifier; RECOMP [7] trains dedicated compressors. All share a structural cost: **a neural forward pass over the prompt at compression time**.

That cost is acceptable inside one application; it is not obviously acceptable at the *LLM gateway*. A gateway such as Mesh API exposes one OpenAI-compatible endpoint fronting 300+ models [15], applies routing, fallback and templating, and processes *every* request for *every* tenant. There a compressor inherits three constraints the literature rarely measures:

- 1. Critical-path latency.** Compression latency adds directly to time-to-first-token. In the LongLLMLingua evaluation, Selective Context's *net* speedup is $0.6\times$ —it makes the request *slower* [5].
- 2. Accelerator footprint.** A multi-tenant, horizontally-scaled gateway paying for a dedicated compressor GPU (or stolen VRAM) multiplies fleet cost.
- 3. Tail behavior.** Neural-compressor latency grows with prompt length and contends with generation for the accelerator, widening p95/p99.

We revisit *classical*, model-free extractive compression—TextRank [8], LexRank [9], centroid [10], MMR [11], BM25 [12]—as a *systems* primitive for the gate. Our central, somewhat counterintuitive finding: at realistic gateway ratios, *query-aware* classical methods do not merely approach the neural baseline—they **exceed** a faithful Selective-Context reimplementation on answer retention, because that method is *query-agnostic* while a gateway request almost always carries an explicit query.

2. Related Work

Neural prompt compression

Selective Context [3] removes low-self-information units. LLMingua [4] adds a budget controller and iterative perplexity pruning; LongLLMingua [5] adds question-aware coarse-to-fine compression, reordering and subsequence recovery; LLMingua-2 [6] swaps perplexity for a distilled classifier (3–6× faster). RECOMP [7] trains compressors for RAG. All require a forward pass at compression time. Notably, in [5] simple lexical baselines (BM25, Gzip) outranked Selective Context on coarse selection—a hint we make systematic.

Classical extractive summarization & ranking

TextRank [8] and LexRank [9] score sentences by graph centrality; centroid methods [10] by proximity to a document centroid; MMR [11] trades relevance against redundancy; BM25 [12] is the canonical lexical query-relevance function.

LLM routing & gateways

Routers like RouteLLM [13] pick a model per query; gateways [15] add a unified API, fallback and templating. We study a complementary transformation—*what context* reaches the model—co-located at the same layer. Co-location is practical because serving stacks (PagedAttention/vLLM [14], FlashAttention) already saturate the GPU with generation; a CPU-side compressor adds no accelerator contention.

3. The Gateway Compression Problem

A request is (q, C) with query q and context C of sentences. A compressor f returns $C' = f(q, C)$ with token budget $|C'| \leq \beta |C|$. The application objective maximizes downstream quality U for a fixed prompt. The gateway objective differs: hold an SLA over a heterogeneous, multi-tenant stream while minimizing fleet cost. With ℓ_f the added latency and m_f the resident accelerator memory:

$$\min_f E[\text{cost}(C')] \text{ s.t. } \ell_f(|C'|) \leq \tau \text{ (critical path)}, m_f \leq \mu \text{ (per-replica VRAM)}, U(q, C') \geq U_{\min}.$$

Neural compressors optimize U aggressively but treat ℓ_f and m_f as free. At the gate they are not. Classical f have $m_f \approx 0$ and ℓ_f of microseconds– milliseconds; the empirical question is whether they meet $U_{\min}$.

4. Method

We evaluate model-free extractive compressors that select a subset of sentences and restore source order, query-aware when q is present (the gateway case): **Lead/Random** baselines; **TF-IDF(+Q)** salience; **TextRank(+Q)** PageRank over a TF-IDF cosine graph with a query personalization vector; **Centroid+MMR** greedy query/centroid relevance vs redundancy; **BM25+Q** Okapi sentence relevance. **Neural baseline**: a faithful Selective Context [3]— per-token NLL under distilgpt2 in a sliding 512-token

window, aggregated to per-sentence self-information; query-agnostic by design, requiring a forward pass over the whole prompt. At the gateway, f runs as a stateless CPU stage between request parsing and model dispatch, adding no GPU contention.

5. Experimental Setup

Benchmark. A multi-document QA set from SQuAD v1.1 [16]: each item concatenates the gold paragraph with 7 distractor paragraphs from unrelated articles, gold placed in the middle to stress "lost-in-the-middle" [2]. 200 items, mean 1307 tokens over 8 passages. **Metrics:** answer retention (compressed context still contains the SQuAD-normalized gold span; intrinsic, LLM-free), token reduction (cl100k_base), latency (median ms/request over {256...4096} tokens), footprint (resident weights). All timing on one Apple M3 Pro CPU; every method, including the neural baseline, is timed on the same hardware.

6. Results

Quality — query-awareness is the discriminator. Every query-aware classical method retains far more answers than every query-agnostic one. TF-IDF+Q (98.5%), TextRank+Q (97.0%), BM25+Q (96.5%) and Centroid+MMR (96.0%) all retain $\approx 96\%$ of gold answers at $\sim 3\times$ compression, versus 38.7% for the neural Selective-Context baseline — which, being query-agnostic, barely beats query-agnostic TF-IDF (54.5%) and the Lead baseline (16.0%). The lesson is not "neural vs classical" but "query-aware vs query-agnostic": a gateway request carries an explicit query, and cheaply conditioning sentence selection on it recovers almost all answer-bearing content. The expensive neural forward pass buys nothing here because Selective Context does not see the query.

Method	Family	Answer ret. (%)	Token red. (%)	Latency (ms)	Footprint (MB)
Lead	classical	16.0	67.2	0.04	≈ 0
Random	classical	38.0	66.4	0.05	≈ 0
TF-IDF	classical	54.5	49.3	0.79	≈ 0
TF-IDF+Q	classical	98.5	51.3	0.99	≈ 0
TextRank	classical	36.0	63.6	1.68	≈ 0
TextRank+Q	classical	97.0	63.9	5.96	≈ 0
Centroid+MMR	classical	96.0	68.7	1.56	≈ 0
BM25+Q	classical	96.5	62.5	0.33	≈ 0
Selective Context (neural)	neural	38.7	66.9	406.32	312

Adding the query signal lifts TF-IDF from 54.5% to 98.5% and TextRank from its query-agnostic score to 97.0% retention — the dominant factor is query-conditioning, not the choice of classical scorer.

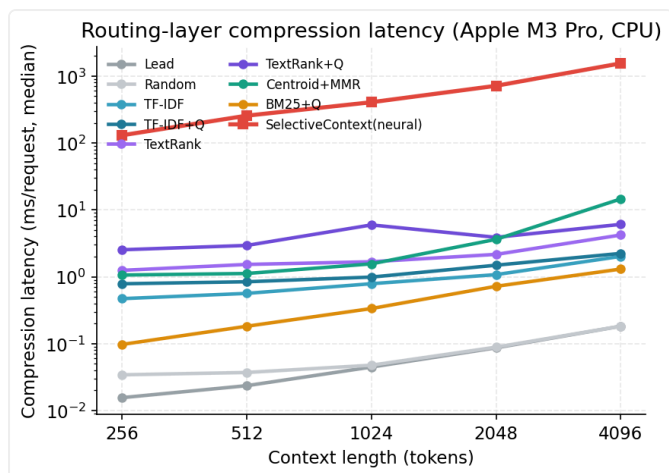


Fig 1. Latency scales with context length; classical methods stay in the single-digit-ms band, the neural compressor in the hundreds of ms (Apple M3 Pro, CPU).

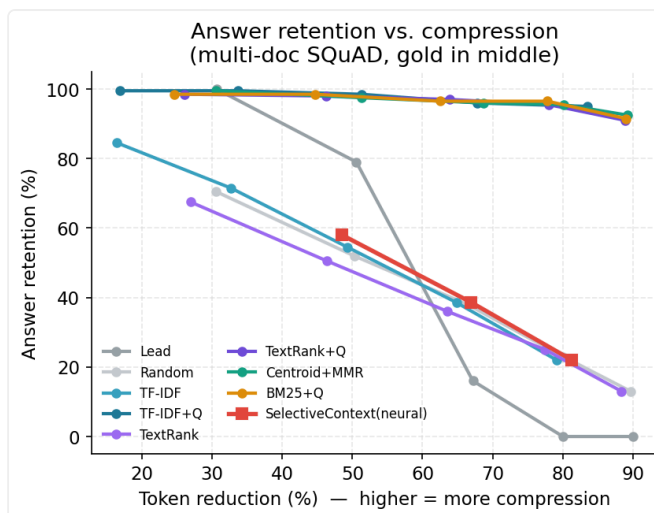


Fig 2. Answer retention vs token reduction. Query-aware lexical methods retain answers far deeper into compression.

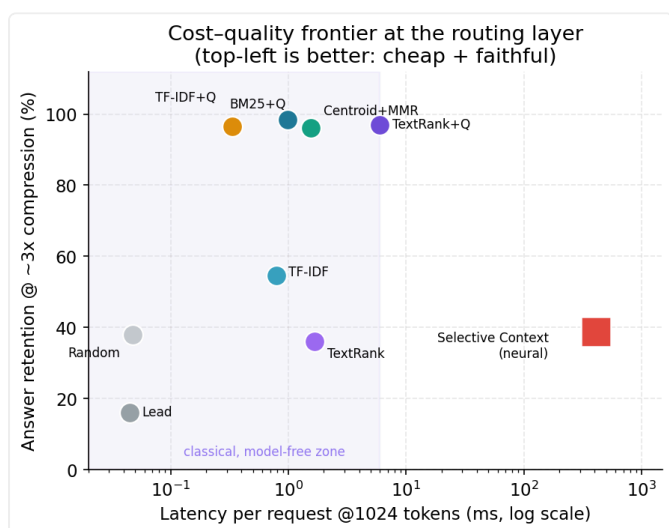


Fig 3. Cost-quality frontier @1024 tokens / $\sim 3\times$ compression. Top-left is better: cheap and faithful.

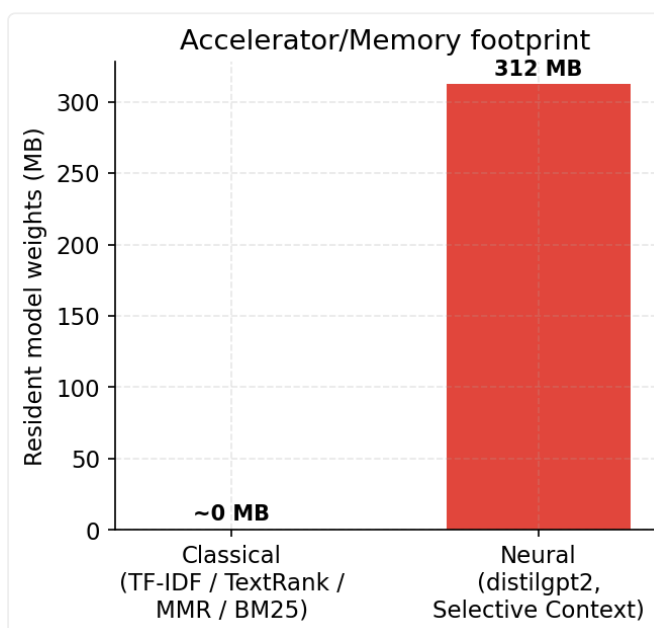


Fig 4. Resident model weights: classical ≈ 0 MB vs 312 MB.

Latency & footprint. Classical methods compress a 1024-token prompt in 0.04–5.96 ms; the neural baseline needs 406 ms (1213 $\times$ slower) and grows with prompt length. The baseline holds 312 MB of weights (81,912,576 params); classical methods hold none.

Fleet-cost projection. At R req/s with mean 1024-token prompts, a neural compressor at 406 ms needs $\approx$ one busy accelerator core per ~ 2.5 req/s plus 312 MB VRAM per replica; the classical stage absorbs the same load on a fraction of a CPU core with no VRAM—the difference between a compression tier that is a rounding error and one that rivals serving itself.

7. Discussion

This is not a claim that classical beats neural compression in general—LongLLMLingua's question-aware perplexity remains stronger for aggressive, abstractive-leaning compression and diffuse answers [5,6]. The claim is narrower: **at the gateway—where latency is on the critical path, footprint multiplies across a fleet, and an explicit query is almost always present—the trade-off inverts, and a**

model-free query-aware extractor is the right default. A natural hybrid runs the classical stage on every request and escalates to a neural compressor only for the few long, query-diffuse prompts a cheap signal flags—mirroring model-router escalation [13].

Limitations. (i) The metric is intrinsic answer-span retention—a strong proxy for extractive faithfulness, not a substitute for end-to-end accuracy (the harness supports an optional gateway-backed eval). (ii) Methods are extractive, sentence-level. (iii) The benchmark is English SQuAD with localized answers; multi-hop/non-English may shift the frontier. (iv) We compare against query-agnostic Selective Context; a query-aware neural compressor would narrow the quality gap at, by construction, even higher latency and footprint.

8. Conclusion

Prompt compression has been studied almost entirely as a modeling problem. Viewed as a *systems* problem at the gateway—the layer that touches every request—the priorities reorder: a compressor must first be cheap in latency and footprint, then maximize retention. In that regime, decades-old query-aware extractive methods retain 96.5% of answers at $\sim 3\times$ compression while running $1213\times$ faster than a neural baseline with zero resident weights. The cheapest context-engineering primitive may also be the best one to put at the gate.

References

1. The Context Engineering Survey: From Prompts to Multi-Agent Architectures. arXiv:2603.09619, 2026.
2. N. F. Liu et al. Lost in the Middle. TACL 2024. arXiv:2307.03172.
3. Y. Li et al. Compressing Context to Enhance Inference Efficiency (Selective Context). EMNLP 2023. arXiv:2304.12102.
4. H. Jiang et al. LLMingua. EMNLP 2023. arXiv:2310.05736.
5. H. Jiang et al. LongLLMingua. ACL 2024. arXiv:2310.06839.
6. Z. Pan et al. LLMingua-2. Findings of ACL 2024. arXiv:2403.12968.
7. F. Xu, W. Shi, E. Choi. RECOMP. ICLR 2024. arXiv:2310.04408.
8. R. Mihalcea, P. Tarau. TextRank. EMNLP 2004.
9. G. Erkan, D. Radev. LexRank. JAIR 2004.
10. D. Radev et al. Centroid-based Summarization. IP&M 2004.
11. J. Carbonell, J. Goldstein. Maximal Marginal Relevance. SIGIR 1998.
12. S. Robertson, H. Zaragoza. BM25 and Beyond. FnTIR 2009.
13. I. Ong et al. RouteLLM. arXiv:2406.18665, 2024.
14. W. Kwon et al. PagedAttention / vLLM. SOSP 2023.
15. Mesh API. Product Overview & Auto-Routing Docs. developers.meshapi.ai, 2026.
16. P. Rajpurkar et al. SQuAD. EMNLP 2016. arXiv:1606.05250.